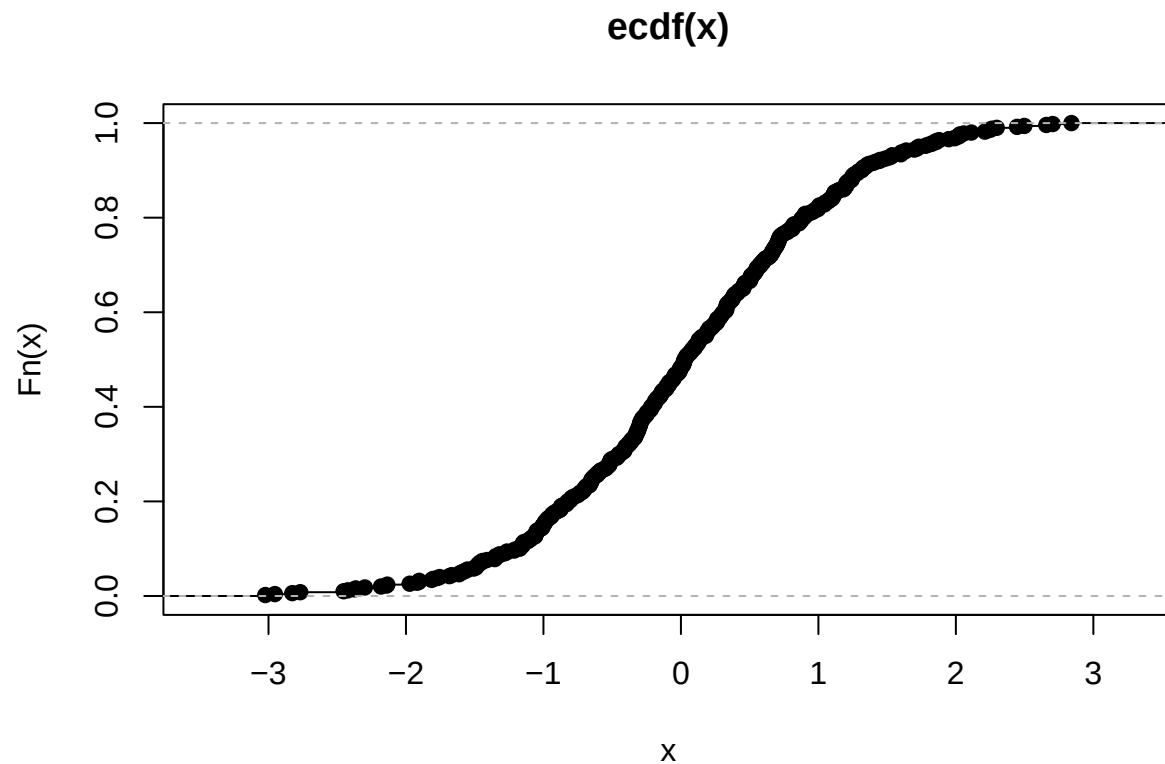


x— title: “week4-1” author: “Greg T.” date: “2026-09-15” output: html_document —

```
# Make an ecdf and plot it:
```

```
x <- rnorm(500,0,1)
Fhat <- ecdf(x)
plot(Fhat)
```



R Markdown

In class work for Week-4

```
n_values <- c(1, 5, 20, 100)
B <- 10000

colors <- c("blue", "green", "orange", "purple")

# Set up an empty plot
plot(
  NA,
  xlim = c(-4, 4),
  ylim = c(0, 1),
  xlab = "x",
  ylab = "CDF",
```

```

    main = "Convergence in Distribution to N(0,1)"
  )

  for (i in 1:length(n_values)) {

    n <- n_values[i]

    x <- rnorm(
      B,
      mean = 0,
      sd = sqrt(1 + 1/n)
    )

    F_hat <- ecdf(x)

    curve(
      F_hat(x),
      from = -4,
      to = 4,
      add = TRUE,
      col = colors[i],
      lwd = 2
    )
  }

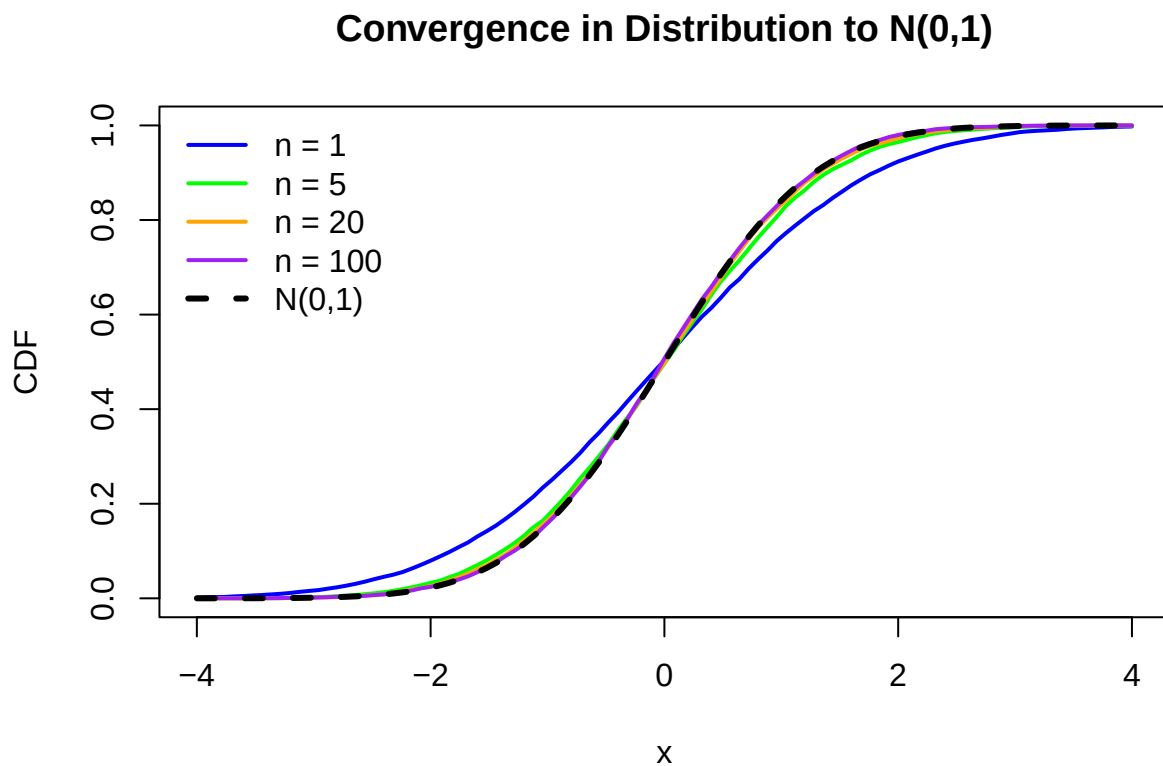
  # Limiting theoretical CDF
  curve(
    pnorm(x),
    from = -4,
    to = 4,
    add = TRUE,
    col = "black",
    lwd = 3,
    lty = 2
  )

  legend(
    "topleft",
    legend = c(
      paste("n =", n_values),
      "N(0,1)"
    ),
    col = c(colors, "black"),
    lwd = c(rep(2, 4), 3),
    lty = c(rep(1, 4), 2),
    bty = "n"
  )

  mtext(
    "Convergence in Distribution to N(0,1)",
    outer = TRUE,
    cex = 1.3
  )

```

)



```
sample_size = c(10,50,100,1000)
n = 100
sim_size = 10000

par(
  mfrow = c(2, 2),
  oma = c(0, 0, 3, 0),
  mar = c(4, 4, 3, 1),
  las = 1
)

for (i in 1:length(sample_size)){
  n <- sample_size[i]
  xbar <- numeric(n) # pre-allocate x_bar, since it's being updated element by element.

  for (j in 1:sim_size){

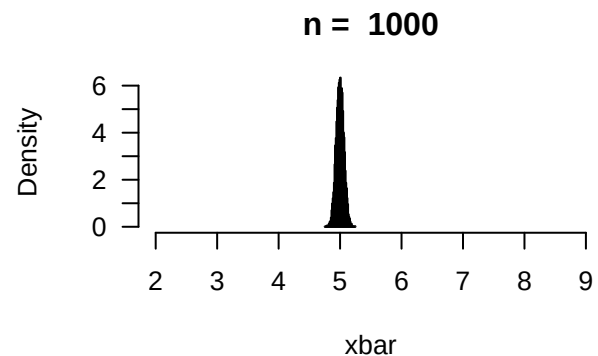
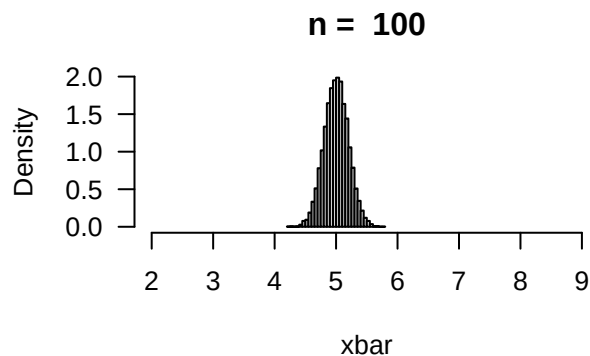
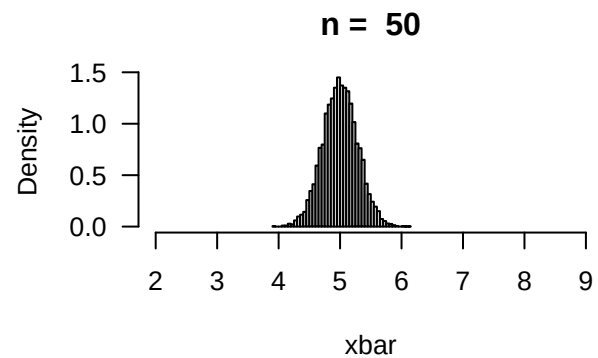
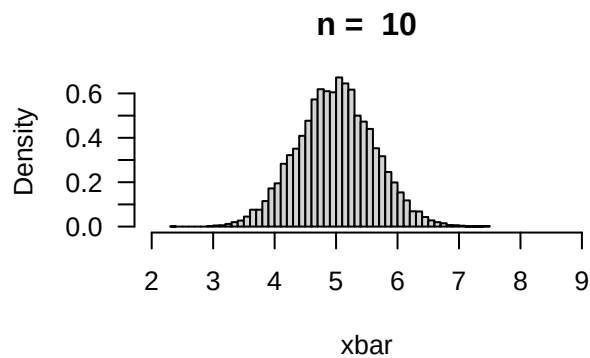
    x <- rnorm(n,5,2)
    xbar[j] <- mean(x)

  }
}
```

```

hist(xbar,
     probability = TRUE,
     breaks = 50,
     xlim = c(2,9),
     main = paste("n = ",n)
)
}

```



Same as above except using the replicate function

```

sample_size <- c(10, 50, 100, 1000)
sim_size <- 10000

par(
  mfrow = c(2, 2),
  oma = c(0, 0, 3, 0),
  mar = c(4, 4, 3, 1),
  las = 1
)

for (i in 1:length(sample_size)) {
  n <- sample_size[i]

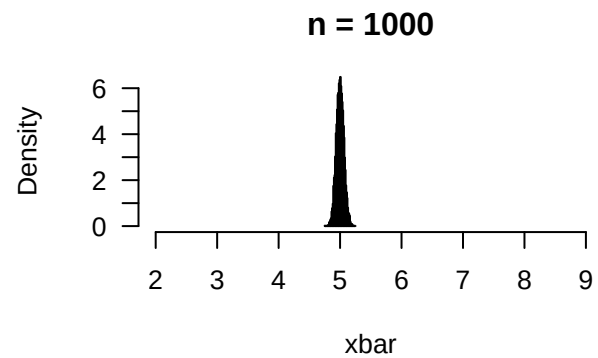
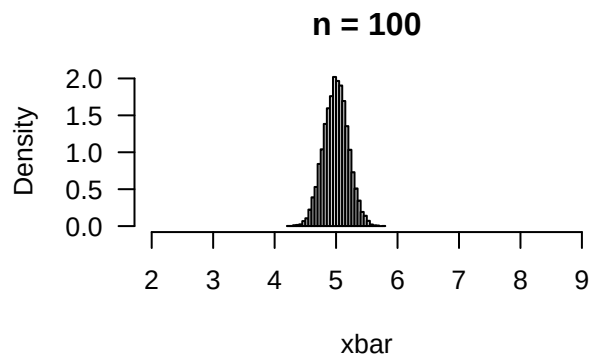
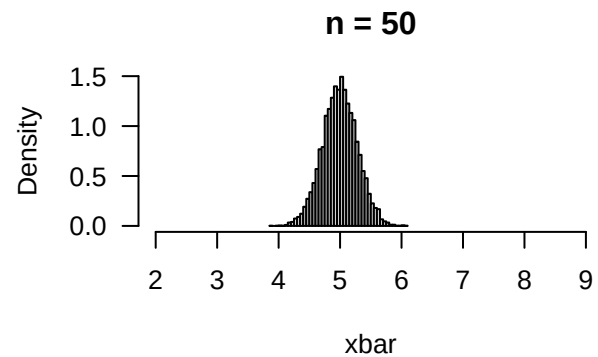
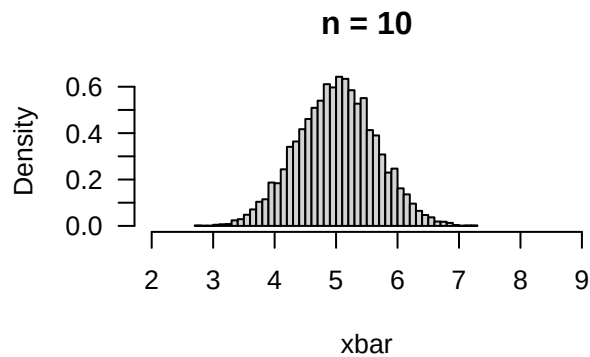
```

```

xbar <- replicate(      # Note, replicate is creating the vector
  sim_size,             # then filling it.
  mean(rnorm(n, 5, 2))
)

hist(
  xbar,
  probability = TRUE,
  breaks = 50,
  xlim = c(2, 9),
  main = paste("n =", n)
)
}

```



```

set.seed(123)

x <- rnorm(1000)
n <- length(x)

# Empirical quantile function
plot(
  (1:n) / n, # plots along the x-axis - normalizes n to 1
  sort(x),   # plots along the y-axis - returns the order statistic of x=rnorm
  type = "s", # draw as a step function
  xlab = "p",

```

```

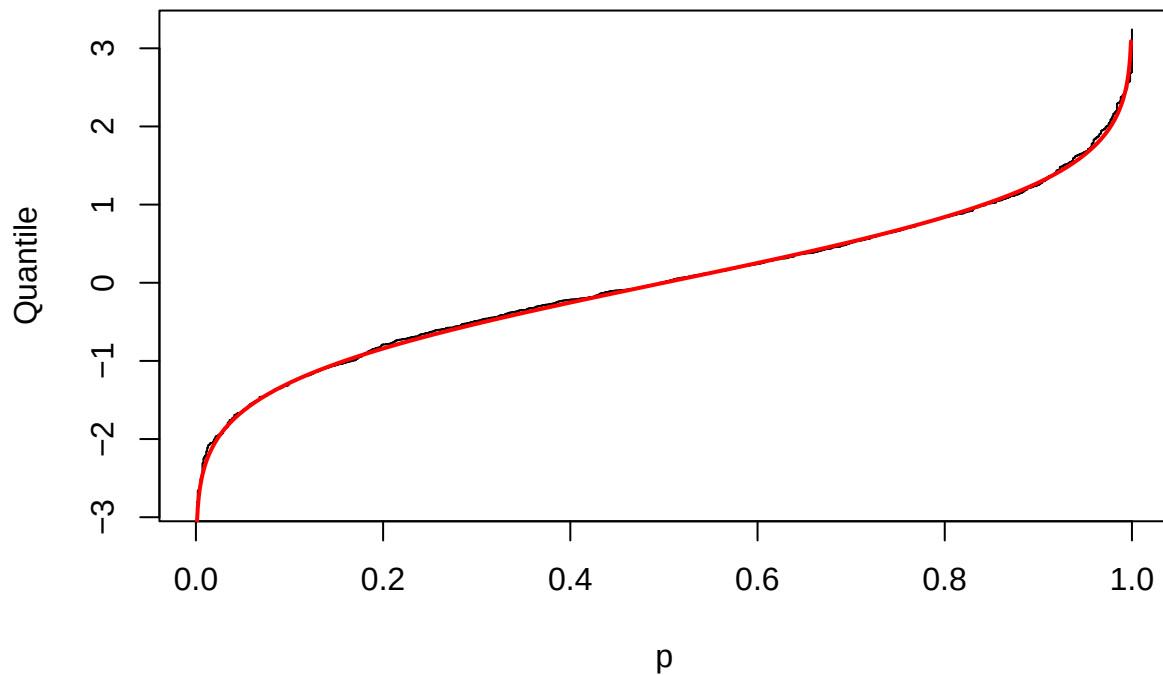
    ylab = "Quantile",
    main = "Empirical vs. Theoretical Normal Quantile Function"
)

# Theoretical standard normal quantile function
p <- seq(0.001, 0.999, length.out = 1000)

lines(
  p,
  qnorm(p),      # apply inverse-phi to the list of quantiles, takes a prob and returns a point on the
  col = "red",
  lwd = 2
)

```

Empirical vs. Theoretical Normal Quantile Function



```

sample_size <- c(1, 2, 4, 5, 10, 20, 50, 100)

epsilon <- 0.2
sim_size <- 10000

# One probability for each sample size
# numeric(length(sample_size)) creates a numeric vector of length 8, initialized w/zeros
# the prob_outlier vector is needed graphing (y-axis)
prob_outlier <- numeric(length(sample_size))
count_outlier <- numeric(length(sample_size))

```

```

#Loop1
# loop over the sample size vector, this is the x-axis in the graph
for (i in 1:length(sample_size)) {

  n <- sample_size[i]

  outlier <- logical(sim_size) # initialize a boolean vector length sim_size

  #Loop2
  # Create the x vector and set the T/F condition based on exceeding epsilon
  for (j in 1:sim_size) {

    x <- rnorm(20, mean = 0, sd = sqrt(2/n))
    x_bar <- mean(x) # could replace with mean(rnorm(n,5,2))

    outlier[j] <- abs(x_bar - 0) > epsilon
  } # end of the simulation loop

  # Store results
  count_outlier[i] <- sum(outlier)
  prob_outlier[i] <- mean(outlier)

  cat(
    "n =", n,
    "count exceeds epsilon =", count_outlier[i],
    " outlier probability =", prob_outlier[i],
    "\n"
  )
} # end of the sample size loop

```

```

## n = 1 count exceeds epsilon = 5265 outlier probability = 0.5265
## n = 2 count exceeds epsilon = 3710 outlier probability = 0.371
## n = 4 count exceeds epsilon = 1949 outlier probability = 0.1949
## n = 5 count exceeds epsilon = 1664 outlier probability = 0.1664
## n = 10 count exceeds epsilon = 452 outlier probability = 0.0452
## n = 20 count exceeds epsilon = 30 outlier probability = 0.003
## n = 50 count exceeds epsilon = 0 outlier probability = 0
## n = 100 count exceeds epsilon = 0 outlier probability = 0

```

```

# Create results table
results <- data.frame(
  n = sample_size,
  count_exceeds_epsilon = count_outlier,
  outlier_probability = prob_outlier
)

# Display table
knitr::kable(
  results,
  digits = 4,
  col.names = c(
    "n",
    "Count",

```

```

    "Probability"
  ),
  caption = "Convergence in Probability Simulation",
  escape = FALSE
)

```

Table 1: Convergence in Probability Simulation

n	Count	Probability
1	5265	0.5265
2	3710	0.3710
4	1949	0.1949
5	1664	0.1664
10	452	0.0452
20	30	0.0030
50	0	0.0000
100	0	0.0000

```

# Plot results
plot(
  sample_size,
  prob_outlier,
  type = "b",
  xlab = "Sample Size (n)",
  ylab = "P(|Xbar - 0| > 0.2)",
  main = "Convergence in Probability"
)

```

Convergence in Probability

